
ОРГАНІЗАЦІЯ БАЗ ДАНИХ ТА ЗНАНЬ

Підзапити в PostgreSQL

У PostgreSQL підзапит (**subquery**) - це запит, який вкладений в інший SQL-запит. Підзапит використовується для отримання даних, які використовуються як вхідні дані для головного запиту. Підзапит може бути використаний в різних частинах SQL-запиту, таких як SELECT, FROM, WHERE, інші операції вибору та фільтрації даних.

Основні характеристики підзапитів в PostgreSQL:

Вкладеність: Підзапит може містити інші підзапити, що дозволяє створювати складні запити з вкладеними логічними операціями.

Кореляція: Підзапит може бути корельованим, тобто він може звертатися до таблиць з зовнішнього запиту, що дозволяє використовувати дані з батьківського запиту.

Типи підзапитів: У PostgreSQL можна використовувати різні типи підзапитів, такі як скалярні підзапити (повертають одне значення), підзапити з вибіркою декількох рядків (повертають набір рядків), підзапити в об'єднаннях, інші.

Використання в WHERE, FROM, SELECT тощо: Підзапити можуть бути використані для фільтрації результатів, визначення вмісту таблиць, обчислення значень і багатьох інших завдань.

Корелюючі підзапити

Підзапити бувають корелюючими і некорелюючими. У прикладах вище команди SELECT виконували фактично один підзапит для всієї команди, наприклад, підзапит повертає мінімальну або середню ціну, яка не зміниться, скільки б ми рядків не вибирали в основному запиті. Результат такого підзапиту не залежав від рядків, які вибираються в основному запиті. І такий підзапит виконується один раз для всього зовнішнього запиту.

Але крім того є **корелюючі підзапити (correlated subquery)**, результати яких залежать від ~~рядків, які витягуються в основному запиті.~~

Корельовані підзапити - це підзапити, які звертаються до зовнішньої таблиці в межах батьківського запиту. Це дозволяє підзапиту використовувати дані з зовнішньої таблиці для обчислення результатів. Основним відмінністю корельованих підзапитів є можливість звертання до даних з батьківського запиту в контексті виконання підзапиту.

Простий приклад корельованого підзапиту в WHERE:

```
SELECT name
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id = emp1
```

У цьому прикладі підзапит корельований, оскільки він звертається до полів зовнішньої таблиці **employees** (конкретно, до **department_id** і **salary**).

```
SELECT name, (SELECT MAX(salary) FROM employees WHERE department_id = employ  
FROM employees;
```

В цьому прикладі підзапит корельований, оскільки він використовує `department_id` з таблиці зовнішнього запиту для обчислення максимальної зарплати в кожному відділі.

Корельовані підзапити дозволяють виконувати більш складні запити, які залежать від даних із зовнішньої таблиці, і вони корисні для обчислень на рядках, що відповідають кожному рядку результату батьківського запиту.

Некорельовані підзапити - це підзапити, які не звертаються до зовнішньої таблиці або не залежать від даних з батьківського запиту.

Вони виконуються незалежно від зовнішнього запиту і повертають набір результатів або одне значення, яке може бути використане в головному запиті.

Основна різниця між корельованими і некорельованими підзапитами полягає в тому, що некорельовані підзапити не використовують дані з контексту батьківського запиту.

Некорельований підзапит в SELECT для обчислення загальної кількості рядків в таблиці:

```
SELECT department_name, (SELECT COUNT(*) FROM  
employees WHERE department_id = departments.id) AS  
employee count FROM departments;
```

Некорельований підзапит в FROM для створення тимчасової таблиці:

```
SELECT d.department name, e.employee name FROM  
employees AS e JOIN (SELECT id, department name FROM  
departments WHERE location = 'New York') AS d ON  
e.department id = d.id;
```

Некорельований підзапит в WHERE для порівняння зі значенням, яке не залежить від зовнішнього запиту:

```
SELECT product name, price FROM products WHERE  
price > (SELECT AVG(price) FROM products);
```

В некорельованих підзапитах немає звернення до полів або даних з батьківського запиту, і вони виконуються незалежно від контексту батьківського запиту.

Вкладений запит у виразі **SELECT** з використанням агрегатної функції:

Припустимо, що у нас є таблиця "products" зі списком товарів та їх цінами. Ми можемо скористатися вкладеним запитом для вибору товарів, ціна яких вища за середню ціну:

sql

 Copy code

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

У цьому прикладі вкладений запит `(SELECT AVG(price) FROM products)` обчислює середню ціну товарів, а головний запит вибирає товари з ціною вище середньої.

Вкладений запит у виразі **FROM** для створення тимчасової таблиці:

Ми можемо використовувати вкладений запит для створення тимчасової таблиці, яку потім використовуємо в головному запиті. Наприклад, виберемо всіх співробітників з певного відділу та їхніх зарплати:

sql

 Copy code

```
SELECT e.name, d.department_name, e.salary
FROM (SELECT id, name, department_id, salary FROM employees WHERE departmen
JOIN departments AS d ON e.department_id = d.id;
```

У вкладеному запиті ми вибрали всіх співробітників з відділу з `department_id = 1`, а головний запит об'єднує цю інформацію з таблицею відділів.

Вкладений запит у виразі WHERE з використанням підзапиту IN:

Виберемо всі продукти, які належать до категорії "Електроніка":

sql

 Copy code

```
SELECT product_name, category
FROM products
WHERE category IN (SELECT category_name FROM categories WHERE category_type
```

У вкладеному запиті ми вибрали всі категорії, які відповідають "Електроніці", а головний запит вибирає товари, які належать до цих категорій.

Вкладені запити дозволяють створювати більш складні та гнучкі SQL-запити, що вимагають обчислень або обробки даних перед вибором результатів.

Використання `ANY` для порівняння з будь-яким значенням у списку:

Приклад: Знайдемо всіх співробітників, які мають зарплату більше, ніж будь-який зі співробітників у відділі "IT".

sql

 Copy code

```
SELECT name, salary
FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 1);
```

У цьому запиті підзапит `(SELECT salary FROM employees WHERE department_id = 1)` повертає всі зарплати співробітників у відділі "IT", і головний запит вибирає співробітників з більшою зарплатою, ніж будь-який зі співробітників у відділі "IT".

Використання `ALL` для порівняння з усіма значеннями у списку:

Приклад: Знайдемо всіх співробітників, які мають зарплату більше, ніж всі співробітники у відділі "HR".

sql

 Copy code

```
SELECT name, salary
FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 2);
```

У цьому запиті підзапит `(SELECT salary FROM employees WHERE department_id = 2)` повертає всі зарплати співробітників у відділі "HR", і головний запит вибирає співробітників, зарплата яких більше, ніж усі співробітники у відділі "HR".

Використання `EXISTS` для перевірки наявності результатів підзапиту:
Приклад: Знайдемо всіх клієнтів, які зробили замовлення в минулому місяці.

```
SELECT customer_name FROM customers c WHERE EXISTS (SELECT  
* FROM orders o WHERE o.customer_id = c.customer_id AND  
o.order_date >= NOW() - INTERVAL '1 month');
```

У цьому запиті підзапит (`SELECT * FROM orders o WHERE o.customer_id = c.customer_id AND o.order_date >= NOW() - INTERVAL '1 month'`) перевіряє наявність замовлень для кожного клієнта протягом останнього місяця, і головний запит вибирає клієнтів, які мають хоча б одне таке замовлення.

Оператори `ANY`, `ALL` і `EXISTS` дозволяють створювати складні запити, що включають в себе підзапити для порівняння, фільтрації та перевірки наявності даних в базі даних PostgreSQL.

Підзапити (subquery) представляють такі запити, які можуть бути вбудовані в інші запити.
Наприклад, визначимо таблиці для товарів, покупців і замовлень:

```
1 CREATE TABLE Products
2 (
3     Id SERIAL PRIMARY KEY,
4     ProductName VARCHAR(30) NOT NULL,
5     Company VARCHAR(20) NOT NULL,
6     ProductCount INTEGER DEFAULT 0,
7     Price NUMERIC NOT NULL
8 );
9 CREATE TABLE Customers
10 (
11     Id SERIAL PRIMARY KEY,
12     FirstName VARCHAR(30) NOT NULL
13 );
14 CREATE TABLE Orders
15 (
16     Id SERIAL PRIMARY KEY,
17     ProductId INTEGER NOT NULL REFERENCES Products(Id) ON DELETE CASCADE,
18     CustomerId INTEGER NOT NULL REFERENCES Customers(Id) ON DELETE CASCADE,
19     CreatedAt DATE NOT NULL,
20     ProductCount INTEGER DEFAULT 1,
21     Price NUMERIC NOT NULL
```

Таблиця Orders містить посилання на дві інші таблиці через поля ProductId і CustomerId.

Додамо в ці таблиці деякі дані:

```
1  INSERT INTO Products(ProductName, Company, ProductCount, Price)
2  VALUES ('iPhone X', 'Apple', 2, 66000),
3  ('iPhone 8', 'Apple', 2, 51000),
4  ('iPhone 7', 'Apple', 5, 42000),
5  ('Galaxy S9', 'Samsung', 2, 56000),
6  ('Galaxy S8 Plus', 'Samsung', 1, 46000),
7  ('Nokia 9', 'HDM Global', 2, 26000),
8  ('Desire 12', 'HTC', 6, 38000);
9
10 INSERT INTO Customers(FirstName)
11 VALUES ('Tom'), ('Bob'), ('Sam');
12
13 INSERT INTO Orders(ProductId, CustomerId, CreatedAt, ProductCount, Price)
14 VALUES
15 (
16     (SELECT Id FROM Products WHERE ProductName='Galaxy S9'),
17     (SELECT Id FROM Customers WHERE FirstName='Tom'),
18     '2017-07-11',
19     2,
20     (SELECT Price FROM Products WHERE ProductName='Galaxy S9')
21 ),
22 (
23     (SELECT Id FROM Products WHERE ProductName='iPhone 8'),
24     (SELECT Id FROM Customers WHERE FirstName='Tom'),
25     '2017-07-13',
26     1,
27     (SELECT Price FROM Products WHERE ProductName='iPhone 8')
28 ),
29 (
30     (SELECT Id FROM Products WHERE ProductName='iPhone 8'),
31     (SELECT Id FROM Customers WHERE FirstName='Bob'),
32     '2017-07-11',
33     1,
34     (SELECT Price FROM Products WHERE ProductName='iPhone 8')
35 );
```

Наприклад, знайдемо товари з таблиці Products, які мають мінімальну ціну:

```
1 SELECT *  
2 FROM Products  
3 WHERE Price = (SELECT MIN(Price) FROM Products);
```

Або знайдемо товари, ціна яких перевищує середню:

1
2
3

```
SELECT *  
FROM Products  
WHERE Price > (SELECT AVG(Price) FROM Products);
```

Data Output Explain Messages Query History					
	id	productname	company	productcount	price
	integer	character varying (30)	character varying (20)	integer	numeric
1	1	iPhone X	Apple	2	66000
2	2	iPhone 8	Apple	2	51000
3	4	Galaxy S9	Samsung	2	56000

Наприклад, виберемо всі замовлення з таблиці Orders, додавши до них інформацію про товар:

```
1 SELECT CreatedAt,  
2     Price,  
3     (SELECT ProductName FROM Products  
4     WHERE Products.Id = Orders.ProductId) AS Product  
5 FROM Orders;
```

Тут для кожного рядка з таблиці Orders буде виконуватися підзапит, результат якого залежить від стовпчика ProductId. І кожен підзапит може повертати різні дані.

```
1 SELECT CreatedAt,  
2     Price,  
3     (SELECT ProductName FROM Products  
4     WHERE Products.Id = Orders.ProductId) AS Product  
5 FROM Orders;
```

Data Output Explain Messages Query History

	createdat date	price numeric	product character varying (30)
1	2017-07-11	56000	Galaxy S9
2	2017-07-13	51000	iPhone 8
3	2017-07-11	51000	iPhone 8

Корелюючий підзапит може виконуватися і для тієї ж таблиці, до якої виконується основний запит. Наприклад, виберемо з таблиці Products ті товари, вартість яких вища за середню ціну товарів для даного виробника:

```
1  SELECT ProductName,  
2         Company,  
3         Price,  
4         (SELECT AVG(Price) FROM Products AS SubProds  
5          WHERE SubProds.Company=Prods.Company) AS AvgPrice  
6  FROM Products AS Prods  
7  WHERE Price >  
8         (SELECT AVG(Price) FROM Products AS SubProds  
9          WHERE SubProds.Company=Prods.Company);
```

	Data Output	Explain	Messages	Query History
	productname character varying (30)	company character varying (20)	price numeric	avgprice numeric
1	iPhone X	Apple	66000	53000.00000000000000
2	Galaxy S9	Samsung	56000	51000.00000000000000

У PostgreSQL, як і в багатьох інших реляційних базах даних, "перегляд" (**View**) є віртуальною таблицею, яка складається з результатів запиту SELECT до одної або декількох реальних таблиць. Перегляд дозволяє створити іменовану інтерфейсну абстракцію для складних або часто використовуваних запитів, щоб легше взаємодіяти з даними та забезпечити рівень абстракції для користувачів бази даних.

Основні характеристики та переваги переглядів в PostgreSQL:

1. **Іменована абстракція.** Перегляд має ім'я та може бути викликаний як будь-яка інша таблиця в запитах SELECT, INSERT, UPDATE і DELETE.
2. **Спрощення запитів.** Перегляди дозволяють створювати складні запити або об'єднувати дані з різних таблиць в одній таблиці. Це спрощує структуру запитів і забезпечує рівень абстракції.

-
- 3. Безпека та доступність.** Перегляд може бути використаний для обмеження доступу до певних рядків або стовпців, приховуючи конфіденційну інформацію від користувачів.
- 4. Можливість оновлення.** Деякі перегляди можуть бути оновлюваними, тобто через них можна вставляти, оновлювати або видаляти дані, якщо це допускається у відповідних таблицях.

Створення перегляду в PostgreSQL виглядає так:

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table1  
WHERE condition;
```

Наприклад, для створення перегляду, який виводить список імен співробітників та їхніх зарплат з таблиці "employees", можна використовувати наступний запит:

```
CREATE VIEW employee_salary_view AS  
SELECT name, salary  
FROM employees;
```

Після створення перегляду **employee_salary_view**, ви можете використовувати його у запитах як звичайну таблицю:

```
SELECT * FROM employee_salary_view;
```

Перегляди - це потужний інструмент для управління та спрощення доступу до даних в PostgreSQL, зокрема в складних або багаторазових запитах.

